

```

// Assignment 1: Parallel BFS and DFS using OpenMP
// Undirected Graph representation using Adjacency List
// =====

#include <iostream>
#include <vector>
#include <queue>
#include <stack>
#include <omp.h>
#include <chrono>
using namespace std;

class Graph {
public:
    int V;
    vector<vector<int>> adj;

    Graph(int v) : V(v), adj(v) {}

    void addEdge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u); // undirected
    }

    // — Sequential BFS —————
    void BFS_Sequential(int start) {
        vector<bool> visited(V, false);
        queue<int> q;
        visited[start] = true;
        q.push(start);

        cout << "\n[Sequential BFS] Visit order: ";
        while (!q.empty()) {
            int node = q.front(); q.pop();
            cout << node << " ";
            for (int nbr : adj[node]) {
                if (!visited[nbr]) {
                    visited[nbr] = true;
                    q.push(nbr);
                }
            }
        }
        cout << endl;
    }

    // — Parallel BFS (level-synchronised) —————
    void BFS_Parallel(int start) {
        vector<bool> visited(V, false);

```

```

vector<int> current_level, next_level;

visited[start] = true;
current_level.push_back(start);

cout << "\n[Parallel BFS] Visit order: ";
while (!current_level.empty()) {
    // Print current level nodes
    for (int node : current_level) cout << node << " ";

    next_level.clear();

    // Process all nodes in current level in parallel
    #pragma omp parallel
    {
        vector<int> local_next;
        #pragma omp for nowait schedule(dynamic)
        for (int i = 0; i < (int)current_level.size(); i++) {
            int node = current_level[i];
            for (int nbr : adj[node]) {
                bool already;
                #pragma omp critical
                {
                    already = visited[nbr];
                    if (!already) visited[nbr] = true;
                }
                if (!already) local_next.push_back(nbr);
            }
        }
        #pragma omp critical
        next_level.insert(next_level.end(), local_next.begin(),
local_next.end());
    }
    current_level = next_level;
}
cout << endl;
}

```

// — Sequential DFS —————

```

void DFS_Sequential(int start) {
    vector<bool> visited(V, false);
    stack<int> st;
    st.push(start);

    cout << "\n[Sequential DFS] Visit order: ";
    while (!st.empty()) {
        int node = st.top(); st.pop();
        if (!visited[node]) {

```

```

        visited[node] = true;
        cout << node << " ";
        for (int i = (int)adj[node].size() - 1; i >= 0; i--)
            if (!visited[adj[node][i]])
                st.push(adj[node][i]);
    }
}
cout << endl;
}

// — Parallel DFS (parallel neighbour exploration) —————
void DFS_Parallel_Helper(int node, vector<bool>& visited) {
    #pragma omp critical
    { visited[node] = true; }
    cout << node << " ";

    #pragma omp parallel for schedule(dynamic)
    for (int i = 0; i < (int)adj[node].size(); i++) {
        int nbr = adj[node][i];
        bool v;
        #pragma omp critical
        { v = visited[nbr]; }
        if (!v) DFS_Parallel_Helper(nbr, visited);
    }
}

void DFS_Parallel(int start) {
    vector<bool> visited(V, false);
    cout << "\n[Parallel DFS] Visit order: ";
    DFS_Parallel_Helper(start, visited);
    cout << endl;
}

};

int main() {
    // Build a sample undirected graph with 8 nodes
    // 0 - 1 - 3 - 7
    // |   |
    // 2   4 - 5
    //     |
    //     6
    Graph g(8);
    g.addEdge(0, 1); g.addEdge(0, 2);
    g.addEdge(1, 3); g.addEdge(1, 4);
    g.addEdge(3, 7); g.addEdge(4, 5);
    g.addEdge(4, 6);

    cout << "=====" << endl;
}

```

```

cout << "   Parallel BFS & DFS using OpenMP       " << endl;
cout << "   Graph: 8 nodes, start node = 0       " << endl;
cout << "===== " << endl;

// — BFS timing —————
auto t1 = chrono::high_resolution_clock::now();
g.BFS_Sequential(0);
auto t2 = chrono::high_resolution_clock::now();
cout << "   Time (Sequential BFS): "
      << chrono::duration_cast<chrono::microseconds>(t2-t1).count() << "
μs\n";

t1 = chrono::high_resolution_clock::now();
g.BFS_Parallel(0);
t2 = chrono::high_resolution_clock::now();
cout << "   Time (Parallel BFS): "
      << chrono::duration_cast<chrono::microseconds>(t2-t1).count() << "
μs\n";

// — DFS timing —————
t1 = chrono::high_resolution_clock::now();
g.DFS_Sequential(0);
t2 = chrono::high_resolution_clock::now();
cout << "   Time (Sequential DFS): "
      << chrono::duration_cast<chrono::microseconds>(t2-t1).count() << "
μs\n";

t1 = chrono::high_resolution_clock::now();
g.DFS_Parallel(0);
t2 = chrono::high_resolution_clock::now();
cout << "   Time (Parallel DFS): "
      << chrono::duration_cast<chrono::microseconds>(t2-t1).count() << "
μs\n";

return 0;
}

```

OUTPUT:

```
User@DESKTOP-U2TPLD0 UCRT64 /c/Users/User/Downloads
$ g++ -fopenmp assignment1_bfs_dfs.cpp -o a1
```

```
User@DESKTOP-U2TPLD0 UCRT64 /c/Users/User/Downloads
$ ./a1
```

```
=====
Parallel BFS & DFS using OpenMP
Graph: 8 nodes, start node = 0
=====
```

```
[Sequential BFS] Visit order: 0 1 2 3 4 7 5 6
Time (Sequential BFS): 329  $\mu$ As
```

```
[Parallel BFS] Visit order: 0 1 2 3 4 7 5 6
Time (Parallel BFS): 1872  $\mu$ As
```

```
[Sequential DFS] Visit order: 0 1 3 7 4 5 6 2
Time (Sequential DFS): 644  $\mu$ As
```

```
[Parallel DFS] Visit order: 0 1 2 3 7 4 5 6
Time (Parallel DFS): 669  $\mu$ As
```

```
User@DESKTOP-U2TPLD0 UCRT64 /c/Users/User/Downloads
$
```